

Κατανεμημένα Συστήματα
Δεύτερη Άσκηση Εργαστηρίου



AM: 161173

Ονοματεπώνυμο: Βεσέλ Νταλίπαϊ

CBInterface.java

Αρχικά φτιάχνουμε το remote interface για το callback ώστε να παίρνει ειδοποίηση όποιος client κάνει εγγραφή με μία μόνο μέθοδο το notification το οποίο δέχεται σαν όρισμα τον τύπο της θέσης και πόσες θέσεις είναι διαθέσιμες.

THInterface.java

Στην συνέχεια φτιάχνουμε το remote interface για την υλοποίηση του συστήματός μας.

Η μέθοδος getSeatByType με παράμετρο τον τύπο της θέσης μας επιστρέφει τον αριθμό που βρίσκεται ο τύπος της θέσης στην λίστα μας.

Η μέθοδος getAvailableSeats μας επιστρέφει έναν πίνακα με όλες τις διαθέσιμες θέσεις που υπάρχουν.

Με την χρήση της μεθόδου bookSeat με παραμέτρους τον τύπο της θέσης, τον αριθμό των θέσεων που θέλει να κλείσει ο πελάτης και το όνομά του κάνουμε μία κράτηση και μας επιστρέφει το κόστος.

Η μέθοδος getGuests μας επιστρέφει μία λίστα με όλες τις κρατήσεις.

Με την μέθοδο cancelSeat με παραμέτρους τον τύπο της θέσης, τον αριθμό των θέσεων και το όνομα του πελάτη κάνουμε ακύρωση της κράτησης του πελάτη και επιστρέφουμε μία λίστα με τις υπόλοιπες θέσεις που έχει μετά την ακύρωση.

Με την μέθοδο registerNotification με παραμέτρους τον τύπο της θέσης και ένα στιγμιότυπο του CBInterface κάνει εγγραφή σε μία λίστα notifications για τον τύπο της θέσης. Σε αντίθεση η μέθοδος unregisterNotification κάνει απεγγραφή.

CBImpl.java

Με το CBImpl υλοποιούμε το remote interface CBInterface.

Δηλώνουμε την μεταβλητή `clientName` όπου θα την χρησιμοποιούσαμε αργότερα στο `notification` που θα στέλναμε στον `client` (αποφάσισα εν τέλει να μην τη χρησιμοποιήσω αλλά ξέχασα να το αφαιρέσω).

Δηλώνουμε και ένα στιγμιότυπο `THInterface` ώστε να μπορούμε να χρησιμοποιήσουμε την μέθοδο `unregisterNotification` αργότερα.

Δημιουργούμε έναν `constructor` με ορίσματα το όνομα του πελάτη και ένα αντικείμενο `THInterface` για την αρχικοποίηση των μεταβλητών μας και καλούμε τον `constructor` από το `UnicastRemoteObject` ώστε να λειτουργεί ως `RMI`.

Με την μέθοδο `notification` εκτυπώνουμε το μήνυμα στον πελάτη και κάνουμε `unregister` από την λίστα ειδοποιήσεων.

THImpl.java

Με το `THImpl` υλοποιούμε το `remote interface THInterface`.

Φτιάχνουμε 3 πίνακες που περιέχουν τους κωδικούς των θέσεων, τις διαθέσιμες θέσεις και τις τιμές τους.

Δημιουργούμε την κλάση `Book` η οποία αντιπροσωπεύει μια κράτηση.

Δηλώνουμε 1 λίστα για τις κρατήσεις και άλλες 5 λίστες για τις ειδοποιήσεις για τον κάθε τύπο θέσης.

Φτιάχνουμε τον `default constructor` ο οποίος καλεί το `UnicastRemoteObject constructor` ώστε να λειτουργεί ως `RMI`.

Η μέθοδος `getSeatsByType` ελέγχει με ένα `loop` σε ποια θέση στην λίστα βρίσκεται ο τύπος που μας δόθηκε και επιστρέφει το αποτέλεσμα. Άμα δεν βρεθεί τότε είναι λάθος και επιστρέφει `-1`.

Η μέθοδος `getAvailableSeat` ελέγχει με ένα `loop` πόσες διαθέσιμες θέσεις υπάρχουν.

Η `bookSeat` πρώτα χρησιμοποιεί την μέθοδο `getSeatsByType` για να βρει το `index` στην λίστα.

Έπειτα ελέγχει άμα βρέθηκε το index, αν ο αριθμός των θέσεων είναι θετικός και αν οι διαθέσιμες θέσεις είναι περισσότερες από αυτές που θέλει να κάνει κράτηση ο χρήστης.

Αφαιρούνται από τις διαθέσιμες θέσεις.

Ελέγχουμε αν υπάρχει ήδη μια τέτοια κράτηση και αν ναι τότε την προσθέτουμε στην ήδη υπάρχουσα.

Προσθέτουμε την κράτηση στην λίστα και γυρνάμε το τελικό ποσό.

Στην μέθοδο getGuests ελέγχουμε για αρχή άμα έχουμε ήδη προσθέσει την κράτηση και αν όχι το προσθέτουμε στη λίστα μας.

Έπειτα ελέγχουμε άμα το όνομα του πελάτη έχει και άλλες κρατήσεις με διαφορετικούς τύπους.

Παίρνουμε το συνολικό κόστος όλων των κρατήσεων του πελάτη και το προσθέτουμε στον άκέραιο total.

Σε περίπτωση που ο πελάτης έχει πάνω από μία κράτηση προσθέτουμε ένα κόμμα στο string.

Φτιάχνουμε το string με την βοήθεια της κλάσης StringBuilder και επιστρέφουμε την λίστα με τα strings.

Η μέθοδος cancelSeat πρώτα ελέγχει άμα ο τύπος και ο αριθμός που της δώσαμε αν υπάρχει και αν είναι θετικός αντίστοιχα.

Ελέγχει άμα υπάρχει το όνομα στις κρατήσεις και αν υπάρχει για αυτόν τον τύπο θέσης.

Έπειτα ελέγχει άμα ο αριθμός που του δώσαμε είναι μικρότερος των θέσεων που υπάρχουν στην κράτηση.

Αν όλα εντάξει αφαιρεί τις θέσεις από την κράτηση και τις προσθέτει στις διαθέσιμες θέσεις.

Άμα τώρα πια οι θέσεις στην κράτηση είναι μηδενικές τότε την αφαιρούμε από την λίστα των κρατήσεων.

Ελέγχουμε για ποιον τύπο θέσης έγινε η ακύρωση και παίρνουμε την λίστα με τις ειδοποιήσεις του.

Έπειτα στέλνουμε ειδοποίηση σε κάθε μέλος της λίστας εκείνου του τύπου με την μέθοδο notification.

Τέλος επιστρέφουμε, σε περίπτωση που υπάρχουν, τις υπόλοιπες κρατήσεις που υπάρχουν στο όνομα που μας δόθηκε.

Με την μέθοδο registerNotification προσθέτουμε τον πελάτη στην λίστα ειδοποιήσεων για τον τύπο που μας δόθηκε. Και με την μέθοδο unregisterNotification τον αφαιρούμε.

THServer.java

Στην αρχή δημιουργούμε ένα νέο στιγμιότυπο της κλάσης THImpl ώστε να έχουμε την λειτουργία του RMI server.

Δημιουργούμε ένα καινούργιο registry στο port 1099.

Στην συνέχεια κάνουμε bind το αντικείμενό μας με το registry και εκτυπώνουμε μήνυμα επιτυχίας.

Σε αντίθετη περίπτωση αντιμετωπίζουμε το exception.

THClient.java

Σε περίπτωση που καλέσουμε το πρόγραμμα χωρίς κάποια παράμετρο εκτυπώνουμε πώς καλούμε το πρόγραμμα.

Παίρνουμε από τις παραμέτρους την εντολή και το hostname που μας παρέχει ο χρήστης.

Περνάμε το stub του server στην δικιά μας κλάση THInterface.

Με ένα switch θα διαχειριστούμε την εντολή που μας δόθηκε.

Σε περίπτωση του list εκτυπώνουμε όλες τις διαθέσιμες θέσεις.

Στην περίπτωση που είναι η εντολή book ελέγχουμε άμα έχουμε το σωστό αριθμό παραμέτρων, παίρνουμε τα στοιχεία που μας έδωσε ο χρήστης (τύπος, αριθμό, όνομα) και πραγματοποιούμε την κράτηση.

Άμα η κράτηση δεν πραγματοποιηθεί, εκτυπώνουμε κατάλληλο μήνυμα στον πελάτη και στην συνέχεια τον ρωτάμε άμα θέλει να κάνει κράτηση για

τις διαθέσιμες θέσεις που υπάρχουν. Σε περίπτωση που ο πελάτης πει ναι γίνεται η κράτηση για τις διαθέσιμες θέσεις.

Σε αντίθετη περίπτωση ρωτάμε τον χρήστη άμα θέλει να κάνει εγγραφή ώστε να λάβει ειδοποίηση όταν υπάρχουν διαθέσιμες θέσεις. Αν ναι κάνει εγγραφή σε λίστα και περιμένει αλλιώς δείχνει κατάλληλο μήνυμα.

Σε περίπτωση που πήγαν όλα καλά την πρώτη φορά που προσπαθήσαμε να κάνουμε κράτηση τότε μας δείχνει κατάλληλο μήνυμα για την κράτηση και το κόστος της.

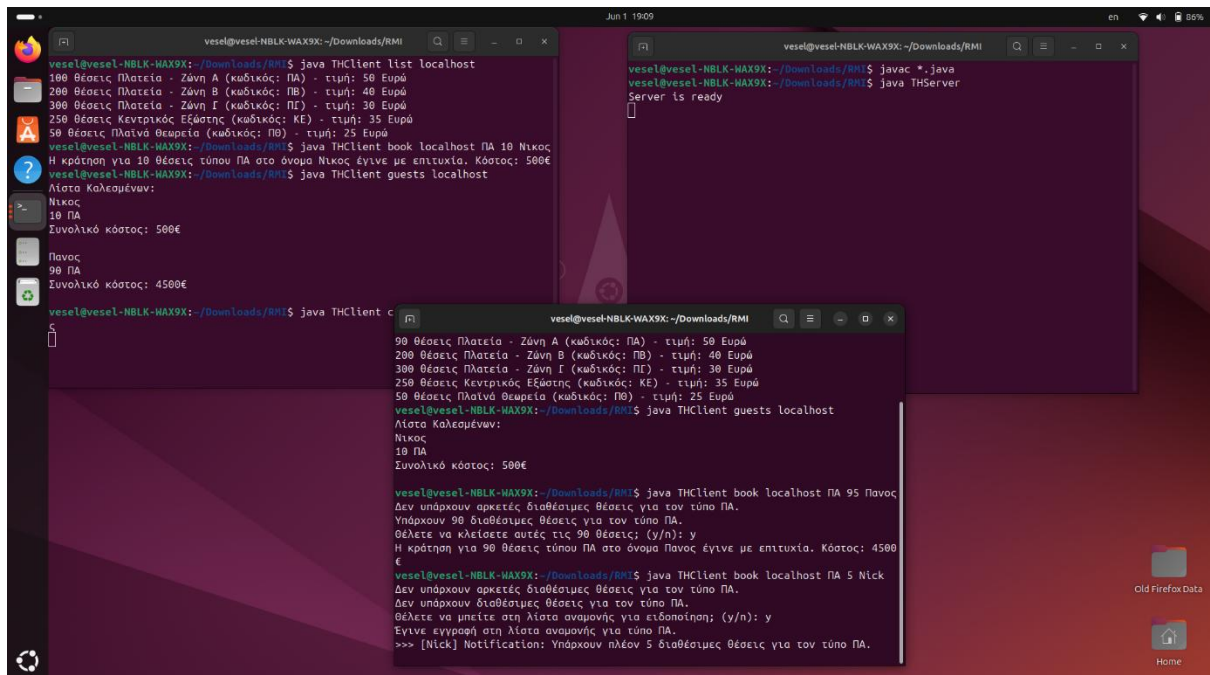
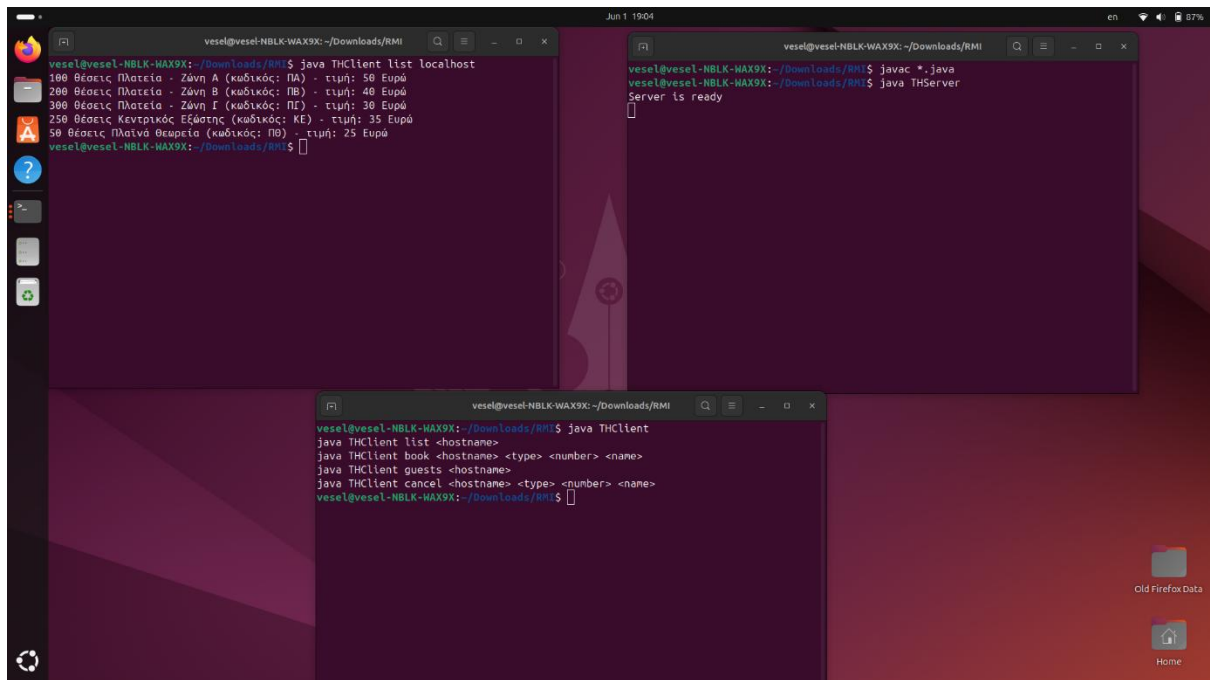
Στην περίπτωση της εντολής guests το πρόγραμμά μας γυρνάει μία λίστα με όλες τις κρατήσεις.

Άμα πάρουμε την εντολή cancel τότε παίρνουμε τις παραμέτρους που μας παρείχε ο χρήστης και κάνουμε την ακύρωση. Σε περίπτωση που δεν βρέθηκε κράτηση ή δεν έχει άλλες κρατήσεις ο πελάτης εκτυπώνουμε κατάλληλο μήνυμα και σε περίπτωση που έχει τις εκτυπώνουμε.

Ενδεικτικά τρεξίματα

Έτρεξα τα προγράμματα σε περιβάλλον Ubuntu καθώς δεν μπορούσα να εμφανίσω τα ελληνικά στα windows 11.

Έκανα compile χρησιμοποιώντας την εντολή javac *.java και στην συνέχεια έτρεξα πρώτα τον Server και μετά τον client.



```
vesel@vesel-NBLK-WAX9X: ~/Downloads/RMI
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient list localhost
100 θέσεις Πλατεία - Ζώνη Α (κωδικός: ΠΑ) - τιμή: 50 Ευρώ
200 θέσεις Πλατεία - Ζώνη Β (κωδικός: ΠΒ) - τιμή: 40 Ευρώ
300 θέσεις Πλατεία - Ζώνη Γ (κωδικός: ΠΓ) - τιμή: 30 Ευρώ
250 θέσεις Κεντρικός Εξώστης (κωδικός: ΚΕ) - τιμή: 35 Ευρώ
50 θέσεις Πλατιά Θερμεία (κωδικός: ΠΘ) - τιμή: 25 Ευρώ
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient book localhost ΠΑ 10 Νίκος
Η κράτηση για 10 θέσεις τύπου ΠΑ στο όνομα Νίκος έγινε με επιτυχία. Κόστος: 500€
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$

vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient
java THClient list <hostname>
java THClient book <hostname> <type> <number> <name>
java THClient guests <hostname>
java THClient cancel <hostname> <type> <number> <name>
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient list localhost
90 θέσεις Πλατεία - Ζώνη Α (κωδικός: ΠΑ) - τιμή: 50 Ευρώ
200 θέσεις Πλατεία - Ζώνη Β (κωδικός: ΠΒ) - τιμή: 40 Ευρώ
300 θέσεις Πλατεία - Ζώνη Γ (κωδικός: ΠΓ) - τιμή: 30 Ευρώ
250 θέσεις Κεντρικός Εξώστης (κωδικός: ΚΕ) - τιμή: 35 Ευρώ
50 θέσεις Πλατιά Θερμεία (κωδικός: ΠΘ) - τιμή: 25 Ευρώ
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient guests localhost
Λίστα Καλεσμένων:
Νίκος
10 ΠΑ
Συνολικό κόστος: 500€

vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$
```

```
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient list localhost
100 θέσεις Πλατεία - Ζώνη Α (κωδικός: ΠΑ) - τιμή: 50 Ευρώ
200 θέσεις Πλατεία - Ζώνη Β (κωδικός: ΠΒ) - τιμή: 40 Ευρώ
300 θέσεις Πλατεία - Ζώνη Γ (κωδικός: ΠΓ) - τιμή: 30 Ευρώ
250 θέσεις Κεντρικός Εξώστης (κωδικός: ΚΕ) - τιμή: 35 Ευρώ
50 θέσεις Πλατιά Θερμεία (κωδικός: ΠΘ) - τιμή: 25 Ευρώ
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient book localhost ΠΑ 10 Νίκος
Η κράτηση για 10 θέσεις τύπου ΠΑ στο όνομα Νίκος έγινε με επιτυχία. Κόστος: 500€
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient guests localhost
Λίστα Καλεσμένων:
Νίκος
10 ΠΑ
Συνολικό κόστος: 500€

vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$

vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient
java THClient list <hostname>
java THClient book <hostname> <type> <number> <name>
java THClient guests <hostname>
java THClient cancel <hostname> <type> <number> <name>
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient list localhost
90 θέσεις Πλατεία - Ζώνη Α (κωδικός: ΠΑ) - τιμή: 50 Ευρώ
200 θέσεις Πλατεία - Ζώνη Β (κωδικός: ΠΒ) - τιμή: 40 Ευρώ
300 θέσεις Πλατεία - Ζώνη Γ (κωδικός: ΠΓ) - τιμή: 30 Ευρώ
250 θέσεις Κεντρικός Εξώστης (κωδικός: ΚΕ) - τιμή: 35 Ευρώ
50 θέσεις Πλατιά Θερμεία (κωδικός: ΠΘ) - τιμή: 25 Ευρώ
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient guests localhost
Λίστα Καλεσμένων:
Νίκος
10 ΠΑ
Συνολικό κόστος: 500€

vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$ java THClient book localhost ΠΑ 95 Πανος
Δεν υπάρχουν αρκετές διαθέσιμες θέσεις για τον τύπο ΠΑ.
Υπάρχουν 90 διαθέσιμες θέσεις για τον τύπο ΠΑ.
Θέλετε να κλείσετε αυτές τις 90 θέσεις; (y/n): y
Η κράτηση για 90 θέσεις τύπου ΠΑ στο όνομα Πανος έγινε με επιτυχία. Κόστος: 4500€
vesel@vesel-NBLK-WAX9X:~/Downloads/RMI$
```